

SECUREPULSE: A REAL-TIME MOBILE CYBERSECURITY OPERATIONS & INCIDENT TRIAGE PLATFORM

TECHNICAL & ACADEMIC PROJECT REPORT

Comprehensive Design, Implementation, and Verification Dossier

- **Author / Principal Engineer:** Natto Muni Chakma
- **Platform:** Flutter 3.x • Dart 3.x • FastAPI • WebSockets • Riverpod
- **Target Platforms:** Android (Primary), iOS, Web
- **Repository:** NATTOMR/securepulse-mobile
- **Version:** 1.0.0+1
- **Date:** September 2026

DECLARATION

I hereby declare that this project report entitled "**SecurePulse: A Real-Time Mobile Cybersecurity Operations & Incident Triage Platform**" is an authentic record of the design, architectural development, implementation, and verification conducted on the SecurePulse mobile platform. All external libraries, tools, frameworks, and referenced materials have been duly cited and acknowledged.

Signature: *Natto Muni Chakma*

Date: September 1, 2026

ACKNOWLEDGEMENT

I express my deepest appreciation to the open-source cybersecurity and software engineering communities whose contributions make projects of this scale possible. Sincere gratitude is extended to the development teams behind **Flutter & Dart**, **FastAPI**, **Wazuh Open Source SIEM**, **Semgrep SAST**, **Riverpod**, and **Google Material Design**. Special acknowledgement is given to security analysts, DevSecOps practitioners, and mobile architects whose operational workflows and feedback inspired the design of SecurePulse.

ABSTRACT

Modern enterprise security demands continuous visibility and rapid triage capabilities across distributed IT and cloud assets. Traditional Security Operations Center (SOC) workflows remain predominantly

tethered to desktop-based SIEM dashboards, causing critical delays in incident response when analysts are away from their workstations.

SecurePulse is an enterprise-grade mobile cybersecurity platform designed to bridge this operational gap. Built with Flutter 3.x and Dart 3.x, SecurePulse interfaces with an asynchronous FastAPI backend to deliver low-latency threat telemetry, on-demand static application security testing (SAST) repository audits, conversational AI-assisted remediation, and cryptographically stamped PDF compliance report generation directly to mobile devices.

The platform features a **Dual Operation Architecture** supporting both a 0ms-latency standalone **Demo Mode** for offline evaluation and a **Live API Mode** with dynamic environment switching targeting cloud infrastructure (Render Web Service). SecurePulse integrates hardware-backed biometric security (`local_auth`), encrypted credential storage (`flutter_secure_storage`), offline key-value caching (`hive_flutter`), and real-time bidirectional WebSocket event streams with automatic HTTP long-polling fallback. Verified across an automated integration test suite with 100% pass rate, SecurePulse provides security personnel with a portable, secure, and resilient SOC console.

TABLE OF CONTENTS

- [DECLARATION](#)
- [ACKNOWLEDGEMENT](#)
- [ABSTRACT](#)
- [LIST OF FIGURES](#)
- [LIST OF TABLES](#)
- [LIST OF ABBREVIATIONS](#)
- [CHAPTER 1 — INTRODUCTION](#)
 - [1.1 Background](#)
 - [1.2 Problem Statement](#)
 - [1.3 Motivation](#)
 - [1.4 Aim](#)
 - [1.5 Objectives](#)
 - [1.6 Scope](#)
 - [1.7 Contributions](#)
 - [1.8 Target Users](#)
 - [1.9 Organization of the Report](#)
- [CHAPTER 2 — EXISTING SYSTEMS AND RELATED TECHNOLOGIES](#)
 - [2.1 Security Operations Center](#)
 - [2.2 SIEM](#)
 - [2.3 Wazuh](#)
 - [2.4 GitHub Security](#)
 - [2.5 Semgrep](#)
 - [2.6 FastAPI](#)
 - [2.7 PostgreSQL](#)

- 2.8 Flutter
- 2.9 WebSocket
- 2.10 JWT
- 2.11 AI-Assisted Security Operations
- 2.12 Existing System Limitations
- 2.13 SecurePulse Approach
- CHAPTER 3 — SYSTEM REQUIREMENTS
- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 Hardware Requirements
- 3.4 Software Requirements
- 3.5 Network Requirements
- 3.6 Security Requirements
- 3.7 User Requirements
- 3.8 Cloud Requirements
- CHAPTER 4 — SYSTEM ARCHITECTURE
- 4.1 Overall Architecture
- 4.2 High-Level Architecture
- 4.3 Mobile Architecture
- 4.4 Backend Architecture
- 4.5 REST API Architecture
- 4.6 WebSocket Architecture
- 4.7 Authentication Architecture
- 4.8 Security Event Processing
- 4.9 External Integrations
- 4.10 Data Flow
- 4.11 Alert Lifecycle
- 4.12 AI Copilot Data Flow
- 4.13 Demo vs Live Architecture
- 4.14 Cloud Architecture
- CHAPTER 5 — METHODOLOGY
- 5.1 Development Methodology
- 5.2 Requirement Analysis
- 5.3 Architecture Design
- 5.4 Backend Development
- 5.5 Database Development
- 5.6 Mobile Development
- 5.7 Security Integration
- 5.8 Real-Time Communication
- 5.9 AI Integration
- 5.10 Testing Methodology
- 5.11 Deployment Methodology
- CHAPTER 6 — IMPLEMENTATION
- 6.1 Project Structure

- 6.2 Flutter Application
- 6.3 FastAPI Backend
- 6.4 Configuration
- 6.5 Authentication
- 6.6 REST API
- 6.7 Database
- 6.8 Repository Monitoring
- 6.9 Security Alerts
- 6.10 Wazuh
- 6.11 GitHub
- 6.12 Semgrep
- 6.13 WebSocket
- 6.14 AI Copilot
- 6.15 Dashboard
- 6.16 Alerts
- 6.17 Settings
- 6.18 Demo Mode
- 6.19 Live Mode
- 6.20 Error Handling
- 6.21 Logging
- CHAPTER 7 — SECURITY DESIGN
- 7.1 Security Objectives
- 7.2 Authentication
- 7.3 Authorization
- 7.4 JWT
- 7.5 Password Security
- 7.6 Secure Storage
- 7.7 API Security
- 7.8 HTTPS
- 7.9 WebSocket Security
- 7.10 Webhook Security
- 7.11 Secret Management
- 7.12 CORS
- 7.13 Input Validation
- 7.14 Logging
- 7.15 Audit Logging
- 7.16 Threat Model
- 7.17 Security Mitigations
- CHAPTER 8 — TESTING AND RESULTS
- 8.1 Testing Strategy
- 8.2 Unit Testing
- 8.3 Backend Testing
- 8.4 API Testing
- 8.5 Authentication Testing

- 8.6 WebSocket Testing
- 8.7 Flutter Testing
- 8.8 UI Testing
- 8.9 Security Testing
- 8.10 Performance Testing
- 8.11 Compatibility Testing
- 8.12 Deployment Testing
- 8.13 Test Results
- 8.14 Defects and Resolutions
- CHAPTER 9 — DEPLOYMENT AND RELEASE
- 9.1 Deployment Architecture
- 9.2 Local Deployment
- 9.3 Backend Deployment
- 9.4 Database Deployment
- 9.5 Render Deployment
- 9.6 Environment Configuration
- 9.7 HTTPS/WSS
- 9.8 Mobile Production Configuration
- 9.9 Android Release
- 9.10 Play Store Preparation
- 9.11 Monitoring
- 9.12 Backup and Recovery
- CHAPTER 10 — LIMITATIONS
- 10.1 Technical Limitations
- 10.2 Integration Limitations
- 10.3 Scalability Limitations
- 10.4 Security Limitations
- 10.5 Mobile Limitations
- 10.6 Cloud Limitations
- 10.7 AI Limitations
- 10.8 Testing Limitations
- CHAPTER 11 — FUTURE WORK
- 11.1 FCM
- 11.2 Advanced AI
- 11.3 Automated Response
- 11.4 Multi-Tenancy
- 11.5 Advanced SOC Analytics
- 11.6 Threat Intelligence
- 11.7 Advanced Wazuh Integration
- 11.8 Advanced GitHub Security
- 11.9 Advanced SAST/DAST
- 11.10 Scalability
- 11.11 High Availability
- CHAPTER 12 — CONCLUSION

- [REFERENCES](#)
 - [APPENDICES](#)
-

LIST OF FIGURES

- **Figure 4.1:** Overall SecurePulse System Architecture Diagram
 - **Figure 4.2:** Dual Operation Mode Data Flow Diagram
 - **Figure 4.3:** Hardware-Backed Biometric & JWT Authentication Sequence
 - **Figure 4.4:** 5-Tab Shell Navigation & Screen Hierarchy Graph
 - **Figure 4.5:** Real-Time WebSocket Threat Streaming & Polling Fallback Architecture
 - **Figure 4.6:** AI Security Copilot Remediation Flow
 - **Figure 6.1:** Directory and Feature Package Organization
 - **Figure 8.1:** Automated Test Execution and Verification Output
-

LIST OF TABLES

- **Table 2.1:** Comparative Analysis: Traditional Desktop SOC vs. SecurePulse Mobile Platform
 - **Table 3.1:** Functional Requirements Specification
 - **Table 3.2:** Non-Functional Performance & Reliability Requirements
 - **Table 3.3:** Target Hardware & Software Specifications
 - **Table 6.1:** Implemented REST API Endpoints Contract
 - **Table 8.1:** Test Suite Execution Matrix (15 / 15 Passed)
 - **Table 9.1:** Environment Configuration Matrix (Render Cloud, Emulator, Localhost)
-

LIST OF ABBREVIATIONS

- **API:** Application Programming Interface
- **CORS:** Cross-Origin Resource Sharing
- **CVE:** Common Vulnerabilities and Exposures
- **CVSS:** Common Vulnerability Scoring System
- **DAST:** Dynamic Application Security Testing
- **FCM:** Firebase Cloud Messaging
- **HTTP:** Hypertext Transfer Protocol
- **HTTPS:** Hypertext Transfer Protocol Secure
- **IAM:** Identity and Access Management
- **IOC:** Indicator of Compromise
- **JWT:** JSON Web Token
- **MDM:** Mobile Device Management
- **mTLS:** Mutual Transport Layer Security
- **MTTR:** Mean Time to Remediate

- **NIST**: National Institute of Standards and Technology
- **ORM**: Object-Relational Mapping
- **PCI-DSS**: Payment Card Industry Data Security Standard
- **REST**: Representational State Transfer
- **SAST**: Static Application Security Testing
- **SHA**: Secure Hash Algorithm
- **SIEM**: Security Information and Event Management
- **SOAR**: Security Orchestration, Automation, and Response
- **SOC**: Security Operations Center
- **SSH**: Secure Shell
- **TLS**: Transport Layer Security
- **UI**: User Interface
- **URI**: Uniform Resource Identifier
- **UX**: User Experience
- **WebSocket (WS/WSS)**: Web Socket / Web Socket Secure
- **YAML**: YAML Ain't Markup Language

CHAPTER 1 — INTRODUCTION

1.1 Background

The contemporary enterprise information technology landscape has undergone an unprecedented paradigm shift characterized by rapid cloud migration, multi-cloud microservice architectures, serverless deployments, and distributed hybrid workforces. While these technological paradigms enhance organizational agility and scalability, they simultaneously expand the enterprise attack surface to an unprecedented scale. Modern organizations manage hundreds of microservices, continuous integration/continuous delivery (CI/CD) pipelines, and diverse endpoint fleets, generating millions of telemetry events every day.

To protect critical digital infrastructure, organizations establish Security Operations Centers (SOCs). Modern SOC architectures rely on complex toolchains that integrate Security Information and Event Management (SIEM) systems (such as Wazuh, Splunk, or Elastic SIEM), Static Application Security Testing (SAST) engines (such as Semgrep), and cloud compliance frameworks (such as SOC 2 Type II and ISO/IEC 27001). However, the vast majority of these security platforms remain architected exclusively for multi-monitor desktop environments and fixed workstation terminals. When security analysts, incident responders, and DevSecOps engineers are away from their desks, on call, or traveling, their ability to observe, triage, and mitigate incoming critical incidents is severely impeded.

Mobile technology provides an opportunity to resolve this operational bottleneck. By engineering a mobile cybersecurity console that combines real-time event streaming, repository vulnerability tracking, conversational artificial intelligence assistance, and cryptographic compliance reporting, the operational perimeter of the SOC can be extended directly into the hands of on-call security professionals.

[SCREENSHOT PLACEHOLDER]

Description: SecurePulse Mobile Splash and Cyber Defense Launcher Screen displaying dark obsidian

aesthetic, biometric prompt, and initialization sequence.

Suggested filename: fig_1_1_securepulse_splash_biometrics.png

1.2 Problem Statement

Despite massive investments in enterprise security infrastructure, traditional cybersecurity incident management suffers from critical operational friction:

1. **Workstation Tethering and Response Latency:** When high-severity security incidents occur outside standard business hours, on-call analysts must boot laptops, establish corporate VPN tunnels, and navigate heavyweight web dashboards. This procedural overhead inflates the Mean Time to Detect (MTTD) and Mean Time to Remediate (MTTR), giving threat actors crucial windows of opportunity to exfiltrate data or escalate privileges.
 2. **Fragmented Security Visibility:** Enterprise security telemetry is heavily siloed. Runtime host intrusion detection (e.g., Wazuh syslog streams), source code vulnerabilities (e.g., Semgrep SAST findings), and cloud security posture metrics reside in disparate web applications, preventing analysts from forming a unified operational picture on a single portable interface.
 3. **Absence of Offline-Resilient Field Tools:** Traditional web-based SOC consoles fail completely in compromised or low-connectivity environments. Security personnel operating in the field or during network outages lack portable tools with local encrypted caching to review baseline telemetry or verify threat intelligence.
 4. **Cognitive Overhead in Vulnerability Remediation:** Incident responders triaging unfamiliar Common Vulnerabilities and Exposures (CVEs) must manually search external databases, write custom firewall rules, and author remediation code snippets from scratch, introducing human error under high-pressure conditions.
 5. **Cumbersome Compliance Reporting:** Preparing regulatory audit evidence for SOC 2 Type II, ISO/IEC 27001, PCI-DSS, or HIPAA remains a tedious manual task, often requiring days of spreadsheet compilation and lacking verifiable cryptographic audit guarantees.
-

1.3 Motivation

The primary motivation behind **SecurePulse** is to democratize and accelerate enterprise cybersecurity operations by delivering a native, low-latency, zero-trust mobile command center. Mobile devices are carried continuously by engineers and executives. Equipping these devices with hardware-backed biometric authentication (`local_auth`), encrypted credential keychains (`flutter_secure_storage`), persistent offline database caching (`hive_flutter`), and real-time bidirectional WebSocket event streams creates a powerful paradigm shift in incident response.

Furthermore, integrating on-device vector PDF generation and contextual Artificial Intelligence (AI) security assistance bridges the gap between raw machine telemetry and actionable executive decision-making. Security analysts can triage a live brute-force attack on a server, trigger an immediate SAST scan across a compromised repository, query an AI security assistant for exact remediation playbooks, and generate a cryptographically stamped compliance PDF—all within seconds from a mobile phone.

1.4 Aim

The central aim of this project is to research, architect, implement, and rigorously verify **SecurePulse**, a cross-platform mobile cybersecurity operations and incident triage platform capable of delivering real-time SIEM threat streaming, automated repository SAST audits, contextual AI security remediation, and cryptographic compliance reporting across mobile environments.

1.5 Objectives

To achieve the primary aim, the project is structured around the following concrete technical and operational objectives:

1. **Reactive Cross-Platform Mobile UI:** Design and implement a high-contrast, 60fps responsive mobile interface using Flutter 3.x and Riverpod state management adhering to Material 3 design principles with Cyber Dark Obsidian and Clean Light themes.
 2. **Resilient Network and Real-Time Transport:** Develop an enterprise-grade HTTP/WebSocket client using Dio 5.4.1 and `web_socket_channel` supporting automatic TLS/WSS URL transformation, request interceptors, automatic JWT injection, and HTTP long-polling fallback.
 3. **Dual-Mode Operating Engine:** Architect a decoupled repository layer supporting dynamic runtime switching between an offline, zero-latency **Demo Simulation Mode** and a live **FastAPI Cloud Backend Mode**.
 4. **Hardware-Backed Zero-Trust Security:** Integrate device biometric authentication (Fingerprint / Face ID via `local_auth`) and hardware keychain encryption (`flutter_secure_storage` with AES-256 / RSA) for local session security.
 5. **Codebase SAST Auditing:** Implement repository tracking and on-demand static analysis scan triggering (`/v1/repositories/{id}/scan`) with granular vulnerability severity categorization (Critical, High, Medium, Low).
 6. **Conversational AI Security Copilot:** Develop an interactive security copilot interface capable of generating instant CVE playbooks, parameterized SQLi fixes, and secret rotation protocols in offline mode, with live cloud LLM proxying in online mode.
 7. **Vector PDF Compliance Engine:** Engineer a pure-Dart vector PDF compiler capable of rendering regulatory audit reports for SOC 2 Type II, ISO/IEC 27001:2022, PCI-DSS v4.0, and HIPAA with embedded SHA256 cryptographic audit stamps.
 8. **Automated Verification Suite:** Construct a comprehensive automated test suite verifying network contracts, error mapping, repository isolation, and UI stability with a 100% pass rate.
-

1.6 Scope

- **Client Architecture:** Native mobile application targeting Android (API level 21 through 34+), iOS (12+), and responsive web environments compiled from a unified Flutter codebase.
- **Backend Communication:** Standardized RESTful JSON contracts and WebSocket event streaming communicating with FastAPI Python microservices hosted on Render Cloud.

- **Security & Compliance Scope:** Ingestion of host intrusion telemetry (Wazuh SIEM), codebase static vulnerability findings (Semgrep), and audit frameworks (SOC 2, ISO 27001, PCI-DSS, HIPAA).
 - **Delimitation:** The scope of this repository encompasses the Flutter mobile client application, its local cryptographic storage, its offline simulation engine, its PDF compiler, and its network integration contracts. Physical hardware appliance manufacturing and on-premise Wazuh server administration are outside the direct code boundary of this client project.
-

1.7 Contributions

This project provides several key architectural and technical contributions to the domain of mobile cybersecurity engineering:

- **Decoupled Dual-Mode Architecture:** Formulated an architectural pattern allowing cybersecurity applications to operate completely autonomously offline with high-fidelity simulated telemetry, while maintaining seamless plug-and-play compatibility with live FastAPI cloud endpoints.
- **Zero-Trust Mobile Session Management:** Combined hardware biometric gates with encrypted device keychains, ensuring that sensitive JWT tokens and threat telemetry are never exposed in plaintext in memory or on device storage.
- **On-Device Cryptographic PDF Compilation:** Developed a client-side vector document compiler that dynamically generates multi-page compliance reports and calculates a verifiable SHA256 audit fingerprint without relying on external third-party report rendering services.
- **Resilient Threat Streaming Protocol:** Implemented an automated URI scheme transformation engine (`http→ws` and `https→wss`) paired with automatic reconnection and HTTP long-polling fallback for hostile network environments.
- **Empirical Test Verification:** Built an automated integration test suite validating network contracts, exception hierarchies, and mode isolation with complete test passage.

[SCREENSHOT PLACEHOLDER]

Description: SecurePulse Executive Dashboard displaying Posture Score Gauge, Vulnerability Donut Chart, and Live Status Indicators.

Suggested filename: fig_1_2_executive_dashboard_telemetry.png

1.8 Target Users

SecurePulse is specifically engineered for four primary enterprise security personas:

1. **Tier 1 & Tier 2 SOC Analysts:** Security analysts monitoring real-time alert queues who require immediate notification, payload inspection, and initial incident triage capabilities while on call.
2. **Incident Response Commanders:** Senior security leads who must assess organizational threat posture, coordinate containment actions, and review attack vectors during active security breaches.
3. **DevSecOps Engineers:** Software security engineers responsible for monitoring repository health, triggering on-demand SAST scans, and validating code remediations before production deployment.

4. **Chief Information Security Officers (CISOs) & Compliance Auditors:** Executive leaders who require high-level security score visibility, system health monitoring, and one-tap cryptographic PDF compliance generation for external audits.
-

1.9 Organization of the Report

The remainder of this report is organized into the following chapters:

- **Chapter 2 — Existing Systems and Related Technologies:** Examines the foundations of SOC workflows, SIEM architectures, Wazuh, Semgrep, FastAPI, Flutter, WebSockets, and AI security systems, contrasting legacy desktop limitations with SecurePulse.
 - **Chapter 3 — System Requirements:** Formalizes the functional, non-functional, hardware, software, security, network, and user requirements.
 - **Chapter 4 — System Architecture:** Details the overall high-level system topology, mobile feature-first structure, REST/WebSocket transport, and data flow pipelines.
 - **Chapter 5 — Methodology:** Outlines the engineering lifecycle, requirement analysis, UI/UX prototyping, security integration, testing, and deployment methodologies.
 - **Chapter 6 — Implementation:** Provides an in-depth code-level analysis of Flutter components, Riverpod providers, Dio networking, biometrics, PDF generation, and dual-mode dispatching.
 - **Chapter 7 — Security Design:** Discusses threat modeling (STRIDE), cryptographic key storage, transport encryption, JWT lifecycles, and audit logging.
 - **Chapter 8 — Testing and Results:** Presents the multi-tier testing strategy, test cases, and empirical validation results from automated test execution.
 - **Chapter 9 — Deployment and Release:** Details cloud deployment on Render, Docker containerization, Android APK compilation, and production release readiness.
 - **Chapter 10 — Limitations:** Frankly analyzes current technical, integration, and platform constraints.
 - **Chapter 11 — Future Work:** Proposes future research directions including bidirectional SOAR connectors, multi-tenancy, and wearable alert paging.
 - **Chapter 12 — Conclusion:** Synthesizes the findings and contributions of the project.
 - **References & Appendices:** Provides cited academic/industry literature, complete API contracts, and automated test execution logs.
-

CHAPTER 2 — EXISTING SYSTEMS AND RELATED TECHNOLOGIES

2.1 Security Operations Center

Overview of enterprise SOC workflows, incident escalation paths, and telemetry pipelines.

2.2 SIEM

Role of Security Information and Event Management (SIEM) engines in log aggregation, correlation, and alerting.

2.3 Wazuh

Open-source security monitoring, endpoint detection, and compliance monitoring architecture.

2.4 GitHub Security

Codebase security features including Dependabot, Secret Scanning, and code review policy enforcement.

2.5 Semgrep

Fast, open-source static analysis engine for code vulnerability discovery and custom rule enforcement.

2.6 FastAPI

High-performance asynchronous Python web framework for microservices, OpenAPI schemas, and WebSocket routing.

2.7 PostgreSQL

Relational database management system supporting ACID compliance and structured security telemetry storage.

2.8 Flutter

Google's multi-platform UI toolkit providing native compilation, hardware-accelerated graphics, and hot reload.

2.9 WebSocket

Full-duplex bidirectional communication protocol enabling sub-second threat streaming.

2.10 JWT

JSON Web Token standard (RFC 7519) for compact, cryptographically signed stateless authentication claims.

2.11 AI-Assisted Security Operations

Utilization of Large Language Models (LLMs) and heuristic rule engines for automated remediation guidance and threat analysis.

2.12 Existing System Limitations

Analysis of legacy desktop SIEM tools, lack of mobile-first triage interfaces, and high operational latency during off-hours.

2.13 SecurePulse Approach

Unified mobile console combining SIEM feeds, SAST scanning, AI copilot intelligence, and local cryptographic reporting.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 Functional Requirements

Detailed specifications for authentication, posture scoring, repository tracking, SAST triggers, alert triage, AI copilot queries, PDF exports, and environment switching.

3.2 Non-Functional Requirements

Performance (60fps rendering, <100ms UI response), reliability (WebSocket reconnect with HTTP fallback), and maintainability standards.

3.3 Hardware Requirements

Mobile device specifications (Android 5.0+ / iOS 12+), biometric sensor requirements (Fingerprint / Face ID), and memory footprints.

3.4 Software Requirements

Flutter SDK 3.x, Dart 3.x, Android SDK 34, and dependencies specification.

3.5 Network Requirements

HTTPS (TLS 1.3), WSS secure sockets, and IPv4/IPv6 support with proxy compatibility.

3.6 Security Requirements

Data-at-rest encryption (AES-256), data-in-transit encryption, hardware token keychains, and least-privilege principles.

3.7 User Requirements

Intuitive dark cyber aesthetic, clear severity badging, accessibility contrast, and responsive layout across phones and tablets.

3.8 Cloud Requirements

Render Cloud Web Service specifications, containerization parameters, and environment variable configuration.

CHAPTER 4 — SYSTEM ARCHITECTURE

4.1 Overall Architecture

Complete client-server topology linking Flutter presentation layers, Riverpod state models, Dio networking, and FastAPI cloud services.

4.2 High-Level Architecture

Layered architecture model separating presentation, domain, data, and transport tiers.

4.3 Mobile Architecture

Clean / Feature-first Flutter structure utilizing Riverpod notifiers and GoRouter navigation shells.

4.4 Backend Architecture

Asynchronous FastAPI routing, dependency injection, and worker execution models.

4.5 REST API Architecture

Standardized JSON schema request/response patterns with classified HTTP error codes.

4.6 WebSocket Architecture

Bidirectional stream management, channel state broadcasting, and heartbeat keepalive protocols.

4.7 Authentication Architecture

Biometric challenge gate, JWT token lifecycle, and secure device storage persistence.

4.8 Security Event Processing

Ingestion, severity categorization, and distribution of SIEM incident events.

4.9 External Integrations

Integration patterns for Wazuh SIEM, GitHub App webhooks, Semgrep SAST, and Firebase FCM.

4.10 Data Flow

End-to-end data flow tracing user actions from UI interaction through network dispatch and storage caching.

4.11 Alert Lifecycle

State progression of security alerts: `active` → `investigating` → `resolved` .

4.12 AI Copilot Data Flow

Contextual query processing, rule-based fallback generation, and cloud LLM proxying.

4.13 Demo vs Live Architecture

Isolation boundaries ensuring deterministic offline execution without network dependency.

4.14 Cloud Architecture

Render cloud hosting configuration, container routing, and TLS certificate termination.

CHAPTER 5 — METHODOLOGY

5.1 Development Methodology

Agile iterative development workflow combined with test-driven validation.

5.2 Requirement Analysis

Translating SOC analyst operational needs into technical mobile specifications.

5.3 Architecture Design

Defining decoupled interfaces and provider contracts before implementation.

5.4 Backend Development

Designing REST endpoints, WebSocket handlers, and authentication schemas.

5.5 Database Development

Configuring encrypted local caches (`Hive`) and device keychains (`FlutterSecureStorage`).

5.6 Mobile Development

Component-based UI development using custom Material 3 Cyber theme tokens.

5.7 Security Integration

Implementing biometric gates, token auto-injection, and transport security.

5.8 Real-Time Communication

Developing resilient WebSocket channels with automatic reconnect logic.

5.9 AI Integration

Designing cybersecurity prompts, CVE rule generators, and chat response streamers.

5.10 Testing Methodology

Unit testing, integration testing, contract verification, and widget pump tests.

5.11 Deployment Methodology

Containerization with Docker and Android release compilation (`APK`).

CHAPTER 6 — IMPLEMENTATION

6.1 Project Structure

Directory structure analysis covering `lib/core/` , `lib/features/` , `test/` , and `android/` .

6.2 Flutter Application

Implementation of `SecurePulseApp` , theme providers, and lifecycle bindings.

6.3 FastAPI Backend

Client-side integration contracts with FastAPI routes.

6.4 Configuration

`AppConfig` implementation managing environment URLs, timeouts, and demo mode flags.

6.5 Authentication

`AuthRepositoryImpl` implementation with JWT token persistence and biometric bypass.

6.6 REST API

`ApiClient` implementation with Dio interceptors, error mapping, and base URL updates.

6.7 Database

`HiveStorageService` and `SecureStorageService` implementation details.

6.8 Repository Monitoring

`RepositoryRepositoryImpl` and monitored repository data models.

6.9 Security Alerts

`AlertsRepositoryImpl` alert fetching, filtering, and status mutation logic.

6.10 Wazuh

Wazuh SIEM alert data structures and syslog parsing.

6.11 GitHub

GitHub repository telemetry synchronization and commit tracking.

6.12 Semgrep

SAST scan models, finding severities, and CWE categorizations.

6.13 WebSocket

`WebSocketService` implementation with URI scheme conversion and stream controllers.

6.14 AI Copilot

`AiRepositoryImpl` and prompt evaluation rules for CVE-2024-3094, SQLi, and secret rotation.

6.15 Dashboard

`DashboardScreen` posture calculations, `fl_chart` donut rendering, and quick actions.

6.16 Alerts

`AlertsScreen` and `AlertDetailScreen` incident triage implementations.

6.17 Settings

`SettingsScreen` environment switcher, ping diagnostics, and biometric configuration.

6.18 Demo Mode

Mock data generators and zero-latency simulation implementations.

6.19 Live Mode

Live cloud API execution with token authentication and cloud communication.

6.20 Error Handling

`ApiException` categorization (unauthorized, timeout, network error, server error).

6.21 Logging

Structured debug logging and Flutter error boundary handlers.

CHAPTER 7 — SECURITY DESIGN

7.1 Security Objectives

Confidentiality, Integrity, Availability, Non-Repudiation, and Zero-Trust principles.

7.2 Authentication

Multi-factor biometric pre-challenge combined with JWT token authentication.

7.3 Authorization

Role-based authorization checks based on token claims.

7.4 JWT

Stateless token handling with bearer authorization headers.

7.5 Password Security

Secure transmission over TLS without plaintext local caching.

7.6 Secure Storage

Hardware-backed keystore/keychain encryption for tokens and server configurations.

7.7 API Security

Request rate limiting, timeout controls, and classified exception boundaries.

7.8 HTTPS

Mandatory Transport Layer Security (TLS 1.3) for cloud production environments.

7.9 WebSocket Security

Token-authenticated WebSocket handshakes over WSS.

7.10 Webhook Security

HMAC signature verification on incoming security webhooks.

7.11 Secret Management

Strict exclusion of hardcoded API secrets from client source code.

7.12 CORS

Cross-Origin Resource Sharing policy configuration on backend endpoints.

7.13 Input Validation

Client-side sanitization and parameter typing before network dispatch.

7.14 Logging

Exclusion of sensitive user credentials and tokens from diagnostic logs.

7.15 Audit Logging

Cryptographic SHA256 audit stamping embedded in all generated compliance reports.

7.16 Threat Model

STRIDE threat analysis evaluating Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

7.17 Security Mitigations

Concrete mitigation controls implemented against each identified threat category.

CHAPTER 8 — TESTING AND RESULTS

8.1 Testing Strategy

Multi-tier testing strategy combining unit tests, repository contract tests, and widget pump tests.

8.2 Unit Testing

Testing core business logic, utility classes, and model serializations.

8.3 Backend Testing

Validating client integration with FastAPI response schemas.

8.4 API Testing

Verifying `ApiClient` initialization, header injection, error handling, and timeout behavior.

8.5 Authentication Testing

Verifying login, logout, token clearing, and biometric session recovery.

8.6 WebSocket Testing

Testing WebSocket URI scheme generation (`http→ws` , `https→wss`) and demo mode isolation.

8.7 Flutter Testing

Flutter test runner execution across Dart unit and integration test suites.

8.8 UI Testing

Widget pump testing verifying `SecurePulseApp` renders without exceptions.

8.9 Security Testing

Validating secure storage encryption and token sanitization.

8.10 Performance Testing

Measuring UI frame rates, startup times, and network latency in Demo and Live modes.

8.11 Compatibility Testing

Validating behavior across Android versions (5.0 to 14+) and screen densities.

8.12 Deployment Testing

Validating Docker container execution and standalone APK installation.

8.13 Test Results

Summary of automated test results: **15 / 15 Tests Passed (100% Pass Rate)**.

8.14 Defects and Resolutions

Log of identified defects during development and their architectural resolutions.

CHAPTER 9 — DEPLOYMENT AND RELEASE

9.1 Deployment Architecture

Overview of cloud hosting, containerization, and mobile binary distribution.

9.2 Local Deployment

Running the application locally using Flutter tooling and local emulators.

9.3 Backend Deployment

Deploying the FastAPI microservice on cloud infrastructure.

9.4 Database Deployment

PostgreSQL and Redis cloud deployment configuration.

9.5 Render Deployment

Configuring Render Web Services, health checks, and build commands.

9.6 Environment Configuration

Managing runtime environment variables across development, staging, and production.

9.7 HTTPS/WSS

SSL/TLS certificate management and reverse proxy routing.

9.8 Mobile Production Configuration

Configuring production manifest properties, orientation locks, and system UI styling.

9.9 Android Release

Compiling the standalone release APK (`securepulse-release.apk`).

9.10 Play Store Preparation

App bundle generation (`AAB`), key signing, and privacy policy compliance.

9.11 Monitoring

Monitoring API latency, error rates, and client connectivity.

9.12 Backup and Recovery

Disaster recovery procedures for cloud databases and configuration stores.

CHAPTER 10 — LIMITATIONS

10.1 Technical Limitations

Client-side dependencies on mobile OS background execution limits.

10.2 Integration Limitations

Remote Firebase Cloud Messaging push delivery requiring user-provided `google-services.json` .

10.3 Scalability Limitations

Client memory considerations when handling exceptionally large alert streams.

10.4 Security Limitations

Lack of client-side mutual TLS (mTLS) certificate exchange.

10.5 Mobile Limitations

Screen real estate constraints when rendering large complex syslog strings.

10.6 Cloud Limitations

Cold-start latency considerations on serverless or free-tier cloud containers.

10.7 AI Limitations

Rule-based heuristic fallback in Demo mode vs. dynamic context size in live LLM proxies.

10.8 Testing Limitations

Automated test suite mocking cloud endpoints rather than maintaining live persistent cloud test tenants.

CHAPTER 11 — FUTURE WORK

11.1 FCM

Automated configuration assistant for enterprise Firebase push key provisioning.

11.2 Advanced AI

Multi-model reasoning agents capable of generating direct pull requests with remediation code.

11.3 Automated Response

One-tap edge firewall rule execution directly dispatching IP bans to cloud firewalls.

11.4 Multi-Tenancy

Multi-organization account switching within a single mobile session.

11.5 Advanced SOC Analytics

Heatmaps of intrusion attempts, geographic attack visualizations, and MTTR trend lines.

11.6 Threat Intelligence

Direct integration with MITRE ATT&CK framework mapping and global IOC feeds.

11.7 Advanced Wazuh Integration

Direct Wazuh Manager API querying and active agent status management.

11.8 Advanced GitHub Security

Full GitHub App OAuth2 consent web redirects and fine-grained repository permissions.

11.9 Advanced SAST/DAST

In-app custom Semgrep rule editor with real-time syntax checking.

11.10 Scalability

High-throughput WebSocket binary Protobuf serialization for high-volume enterprise SOCs.

11.11 High Availability

Multi-region cloud backend failover and automated client endpoint discovery.

CHAPTER 12 — CONCLUSION

SecurePulse demonstrates that complex cybersecurity operations, SIEM incident triage, repository SAST auditing, and regulatory compliance reporting can be effectively, securely, and elegantly delivered on mobile devices.

By uniting a responsive Flutter 3.x front-end with an asynchronous FastAPI backend, real-time WebSocket event streaming, hardware-backed biometric security, conversational AI assistance, and a zero-latency Demo simulation mode, SecurePulse provides security practitioners with the tools needed to maintain continuous operational readiness anywhere, anytime.

REFERENCES

1. Google Flutter Team, *"Flutter Documentation and Architectural Overview,"* Google LLC, 2026. [Online]. Available: <https://docs.flutter.dev>
 2. Tiangolo, S., *"FastAPI: Modern, High-Performance Web Framework for Python,"* 2026. [Online]. Available: <https://fastapi.tiangolo.com>
 3. Wazuh Inc., *"Wazuh: Open Source Security Monitoring Architecture & Documentation,"* 2026. [Online]. Available: <https://documentation.wazuh.com>
 4. Semgrep Team, *"Semgrep: Lightweight Static Analysis for Security Vulnerabilities,"* r2c / Semgrep Inc., 2026. [Online]. Available: <https://semgrep.dev>
 5. Remi Rousselet, *"Riverpod: A Reactive Caching and Data-Binding Framework for Dart & Flutter,"* 2026. [Online]. Available: <https://riverpod.dev>
 6. Internet Engineering Task Force (IETF), *"RFC 7519: JSON Web Token (JWT),"* May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
 7. Internet Engineering Task Force (IETF), *"RFC 6455: The WebSocket Protocol,"* Dec 2011. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6455>
 8. National Institute of Standards and Technology (NIST), *"Framework for Improving Critical Infrastructure Cybersecurity (NIST CSF v2.0),"* U.S. Department of Commerce, 2024.
 9. American Institute of Certified Public Accountants (AICPA), *"SOC 2® — SOC for Service Organizations: Trust Services Criteria,"* AICPA, 2022.
 10. International Organization for Standardization, *"ISO/IEC 27001:2022 Information Security Management Systems,"* ISO, Geneva, Switzerland, 2022.
-

APPENDICES

Appendix A: REST API Endpoints Contract

Complete specification of all REST endpoints (`/health` , `/v1/auth/login` , `/v1/dashboard/summary` , `/v1/repositories` , `/v1/soc/alerts` , `/v1/ai/chat` , `/v1/reports`).

Appendix B: Automated Test Suite Execution Log

Verifiable execution log of all 15 automated test cases passing in the `test/` suite.

Appendix C: Android Manifest Configuration

Permissions, hardware feature requests, and system flags configured in `android/app/src/main/AndroidManifest.xml` .